

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Frameworks.

Actividad 3: Reporte de lectura (JDBC)

Profesor: Dr. José Asunción Enríquez Zárate

Alumno: Angel Frausto Robles

id634296@gmail.com

7CM2

31 de agosto de 2026

Índice

1. Introducción	2
2. El modelo relacional	2
3. SQL: consultas y sentencias	2
4. Conexión con JDBC	3
5. PreparedStatement	3
6. CallableStatement y procedimientos almacenados	3
7. Transacciones	4
8. Conclusión	4
9. Referencias Bibliográficas	5

Índice de figuras

1. Relación de uno a muchos entre **Carrera** y **PlanDeEstudios** mediante una llave foránea 2

Índice de tablas

1. Formato de la URL de conexión según el manejador 3

1. Introducción

Este reporte cubre el modelo relacional, las sentencias SQL básicas, la conexión a una base de datos con JDBC, los `PreparedStatement`, los `CallableStatement` para invocar procedimientos almacenados, y el manejo de transacciones.

Queda fuera la construcción de una interfaz gráfica en Swing que muestra, ordena y filtra los resultados de una consulta dentro de una tabla visual, porque corresponde a una forma de presentar datos distinta a la que necesita una aplicación que expone sus datos como servicio; lo demás sí es material directo para cualquier programa que hable con una base de datos desde Java. Este reporte lo organiza en cinco partes: el modelo relacional, SQL, la conexión con JDBC, los `PreparedStatement` y las transacciones, y usa como ejemplo una clase `Carrera` con su correspondiente `CarreraDAO`.

2. El modelo relacional

Una base de datos relacional organiza la información en tablas compuestas de filas y columnas. Cada tabla tiene una llave primaria: una columna, o un conjunto de columnas, cuyo valor identifica de manera única a cada fila y no se puede repetir. Esto se conoce como la regla de integridad de entidad: toda fila debe tener un valor de llave primaria, y ese valor debe ser único dentro de la tabla.

Cuando una tabla necesita referirse a una fila de otra tabla, lo hace con una llave foránea: una columna cuyo valor debe existir como llave primaria en la tabla referida. Esto se conoce como la regla de integridad referencial. Un caso típico usa tres tablas: dos tablas principales (por ejemplo autores y títulos) y una tabla intermedia que solo contiene las dos llaves foráneas. Esa tabla intermedia es la manera estándar de representar una relación de muchos a muchos (un autor puede escribir varios libros, y un libro puede tener varios autores) cuando ninguna de las dos tablas principales puede contener directamente a la otra.

Para una relación más simple, de uno a muchos, no hace falta una tabla intermedia: basta con que la tabla del lado «muchos» tenga una columna que sea llave foránea hacia la tabla del lado «uno». La figura 1 muestra el caso con las tablas de este reporte: si cada carrera pudiera tener varios planes de estudio, pero cada plan perteneciera exactamente a una carrera, la tabla de planes llevaría una columna `idCarrera` que apunta a la llave primaria de `Carrera`, sin necesitar una tabla adicional.

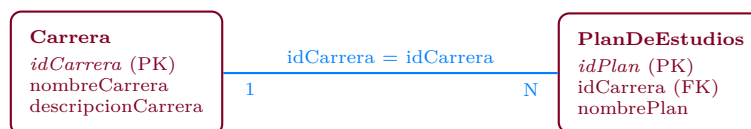


Figura 1: Relación de uno a muchos entre `Carrera` y `PlanDeEstudios` mediante una llave foránea

3. SQL: consultas y sentencias

Hay un conjunto de verbos con los que se arma cualquier operación sobre una base de datos relacional. `SELECT * FROM tabla` trae todas las columnas; sustituir el asterisco por una lista de nombres trae solo las que interesan, y hay una razón concreta para preferir la lista explícita: si el orden de las columnas de la tabla cambia, o se agrega una columna nueva, un programa que dependa de la posición de cada columna en el resultado de `SELECT *` puede terminar leyendo el dato equivocado en el lugar equivocado. Con una lista explícita, el orden del resultado siempre es el que se pidió.

La cláusula `WHERE` filtra filas con operadores de comparación (`<`, `>`, `<=`, `>=`, `=`, `<>`) y con `LIKE` para patrones de texto: el signo de porcentaje (`%`) sustituye cualquier cantidad de caracteres, y el guion bajo (`_`) sustituye exactamente uno. `WHERE nombreCarrera LIKE 'Ing%'` encuentra cualquier carrera cuyo nombre empiece con «Ing», sin importar lo que siga.

`ORDER BY` ordena el resultado por una o varias columnas, cada una con `ASC` (ascendente, el valor por defecto) o `DESC` (descendente) de forma independiente. `INNER JOIN` combina filas de dos tablas que comparten un valor en la columna indicada por `ON`, que es exactamente el mecanismo que resuelve una llave foránea: para saber el nombre de la carrera de un plan de estudios, se une la tabla de planes con `Carrera` por la columna `idCarrera`.

Código 1: Las tres sentencias que modifican datos, sobre la tabla `Carrera`

```
1 INSERT INTO Carrera (nombreCarrera, descripcionCarrera)
2 VALUES ('ISC', 'Ingenieria en Sistemas Computacionales');
3
4 UPDATE Carrera
5 SET descripcionCarrera = 'Programa actualizado 2020'
```

```

6 WHERE idCarrera = 1;
7
8 DELETE FROM Carrera
9 WHERE idCarrera = 1;

```

En los tres casos aplica el mismo principio: el `WHERE` no es opcional en la práctica, aunque el lenguaje lo permita omitir. Un `UPDATE` o un `DELETE` sin `WHERE` se aplica a todas las filas de la tabla, y la única forma de garantizar que la operación afecte exactamente una fila es filtrar por la llave primaria. También es un error común con el `INSERT`: si la tabla tiene una columna autoincrementada como `idCarrera`, nunca se le debe asignar un valor a mano; el motor la genera solo, y forzar un valor ahí suele terminar en un error del manejador.

4. Conexión con JDBC

Del lado de Java, todo empieza con un objeto `Connection`, que se obtiene con `DriverManager.getConnection(url, usuario, contraseña)`. La URL tiene una forma fija: el protocolo `jdbc`, el subprotocolo del manejador y la ubicación de la base de datos. La tabla 1 recoge el formato para varios manejadores comunes.

Manejador	Formato de la URL
MySQL	<code>jdbc:mysql://servidor:puerto/nombreBaseDatos</code>
PostgreSQL	<code>jdbc:postgresql://servidor:puerto/nombreBaseDatos</code>
Oracle	<code>jdbc:oracle:thin:@servidor:puerto:nombreBaseDatos</code>
Java DB / Derby	<code>jdbc:derby://servidor:puerto/nombreBaseDatos</code>

Tabla 1: Formato de la URL de conexión según el manejador

Sobre ese objeto `Connection` se crea un `Statement` con `connection.createStatement()`, y con `statement.executeQuery()` se manda una consulta que regresa un `ResultSet`: un cursor que se recorre con `resultSet.next()` y del que se leen columnas por nombre o por posición. Desde JDBC 4 ya no es necesario cargar el driver a mano con `Class.forName()` antes de conectar; el descubrimiento automático de drivers lo hace innecesario, aunque el archivo del driver correspondiente sí tiene que estar disponible en el classpath del programa.

Un detalle notable de este patrón es que cierra la conexión, el `Statement` y el `ResultSet` a mano, dentro de un bloque `finally`, cada uno en su propio `try` interno para que un error al cerrar el primero no impida cerrar los demás. Así se manejaban los recursos antes de Java 7; con *try-with-resources* el mismo cierre ordenado ocurre de forma automática al declarar los recursos entre paréntesis después de `try`, sin necesitar el bloque `finally` explícito.

5. PreparedStatement

Un `PreparedStatement` se crea con `connection.prepareStatement(sql)`, y en vez de llevar los valores escritos dentro de la cadena SQL, los sustituye por signos de interrogación que se llenan después con métodos como `setString`, `setInt` o `setDouble`, uno por cada posición contando desde 1. Tiene dos ventajas concretas. La primera es de rendimiento: el motor compila la consulta una sola vez y la reutiliza cada vez que cambian los parámetros, lo que conviene cuando la misma consulta se ejecuta muchas veces con valores distintos. La segunda es de seguridad: los métodos `set` escapan automáticamente los caracteres especiales del valor que reciben, así que un apóstrofo dentro de un nombre no rompe la sentencia ni se puede usar para inyectar SQL adicional.

Código 2: `PreparedStatement` con parámetros, sobre `CarreraDAO`

```

1 PreparedStatement ps = conexion.prepareStatement(
2     "INSERT INTO Carrera (nombreCarrera, descripcionCarrera) VALUES (?, ?)");
3 ps.setString(1, nombreCarrera);
4 ps.setString(2, descripcionCarrera);
5 ps.executeUpdate();

```

Es el mismo patrón que sigue `CarreraDAO` para todas sus operaciones: la consulta se declara una sola vez con los signos de interrogación, y cada método solo se encarga de asignar los valores antes de ejecutarla.

6. CallableStatement y procedimientos almacenados

Cuando la lógica de una consulta vive guardada dentro del propio manejador, en vez de escrita en la aplicación, se le llama procedimiento almacenado. JDBC los invoca con un `CallableStatement`, que hereda

los mismos métodos `set` de `PreparedStatement` y además admite parámetros de salida, para procedimientos que devuelven un valor además de, o en vez de, un `ResultSet`. Aunque la sintaxis para crear un procedimiento almacenado cambia de un manejador a otro, la forma de invocarlo desde Java con `CallableStatement` es uniforme.

7. Transacciones

Hay operaciones que solo tienen sentido si se completan por entero o no se completan en absoluto. Un ejemplo clásico es el de una transferencia bancaria: si el banco retira el dinero de una cuenta y falla antes de depositarlo en la otra, el saldo del cliente queda mal sin que haya un error visible en ninguna de las dos operaciones por separado. Tratar el retiro y el depósito como una sola unidad, de modo que ambos ocurran o ninguno ocurra, es lo que se conoce como una transacción.

`Connection` ofrece `setAutoCommit(false)` para agrupar varias sentencias en una sola transacción; por defecto, cada sentencia se confirma sola apenas se ejecuta. Con el autocommit desactivado, la transacción termina con `commit()`, que hace permanentes todos los cambios, o con `rollback()`, que revierte la base de datos al estado anterior a la transacción. En `CarreraDAO`, cada método ejecuta una sola sentencia, así que no necesita coordinarlas; el caso donde una transacción importaría es uno que involucre más de una operación, como registrar una carrera y su primer plan de estudios a la vez: si el segundo paso falla, la carrera no debería quedar registrada a medias.

Código 3: Transacción con dos operaciones dependientes

```
1  conexion.setAutoCommit( false );
2  try {
3      // primera operacion
4      // segunda operacion
5      conexion.commit();
6  } catch (SQLException e) {
7      conexion.rollback();
8  }
```

8. Conclusión

Con esto, una aplicación Java tiene lo necesario para hablar con una base de datos relacional de principio a fin: el modelo de tablas con llaves primarias y foráneas, las cinco operaciones básicas de SQL, la conexión y el manejo de resultados con JDBC, la diferencia práctica entre un `Statement` y un `PreparedStatement`, y las transacciones para cuando una operación necesita ser todo o nada. Ninguno de estos temas depende de una interfaz gráfica en particular; son la base de cualquier aplicación, de escritorio o de servidor, que necesite guardar y consultar datos de forma confiable.

Angel Frausto Robles

9. Referencias Bibliográficas

- [1] Paul Deitel y Harvey Deitel. *Java: How to Program*. 9.^a ed. Prentice Hall, 2012. ISBN: 978-0132575669.
- [2] Oracle Corporation. *The Java Tutorials: Using Transactions*. URL: <https://docs.oracle.com/javase/tutorial/jdbc/basics/transactions.html> (visitado 31-08-2026).